

Алгоритм выстраивания роя в линию по двум якорям (геометрия отрезка и закон управления в симуляторе)

По коду `Drone_Swarm_Simulator_v2/scenarios/linear_chain2.py`

Аннотация

Рассматривается упрощённая постановка: два концевых борта задают концы желаемой линии; остальные дроны выравниваются на прямую, проходящую через эти точки, и стремятся к равномерному распределению вдоль отрезка между ними. Дополнительно используется локальная нелинейная поправка по ближайшим видимым соседям в базисе «вдоль/поперёк» цепи — как вспомогательный стабилизирующий вклад, а не как описание обхода коридора. Ниже зафиксированы обозначения, геометрия, целевые точки якорей и итоговый вектор команды в общей горизонтальной плоскости NED с соответствием именам в исходном коде.

1 Постановка задачи и обозначения

Пусть в эксперименте участвует $N \geq 4$ однотипных мультикоптеров. Два из них играют роль *якорей-концов* цепи: в коде это борты с минимальным и максимальным идентификатором в конфигурации (`endpoint_left_id`, `endpoint_right_id`). Остальные — *внутренние* агенты.

Горизонтальные координаты для логики цепи задаются в *общей* системе NED плоскости Oxy : ось x — север, ось y — восток (как в `LOCAL_POSITION_NED`). Для согласования визуального разнесения бортов в симуляторе к координате y каждого дрона добавляется поправка, зависящая только от его номера (`_did_offset_y`); далее под $p_i = (x_i, y_i)^\top$ понимается уже скорректированное положение в этой общей плоскости.

Состояние концевых дронов в каждый момент принимается за текущие положения концов желаемого отрезка. Обозначим $A(t)$ — положение левого якоря, $B(t)$ — правого (в общих координатах). Внутренние агенты не задают глобальную траекторию коридора; их задача — согласоваться с прямой AB и промежуточными положениями вдоль неё.

В теоретических работах о саморазвертывании между двумя границами часто фигурируют расстояния до ближайших соседей слева и справа и монотонная функция от этих расстояний, обеспечивающая в целевом режиме почти равномерный шаг на интервале между опорными точками. Реализация в `linear_chain2` опирается на ту же идею *локальных* соседей, но основной упор в управлении делается на явную геометрию отрезка AB .

2 Геометрия двух якорей: отрезок AB , базис $(\mathbf{u}, \mathbf{n})$, проекция s_i

Единичный вектор вдоль цепи от левого якоря к правому задаётся нормировкой разности положений:

$$\mathbf{u} = \frac{B - A}{\|B - A\|}, \quad \mathbf{u} \in \mathbb{R}^2, \quad \|\mathbf{u}\| = 1. \quad (1)$$

Вектор $\mathbf{u} = (u_x, u_y)^\top$ строится в коде как `_unit_vector_xy(pos_l, pos_r)`.

Перпендикуляр к $\mathbf{u}$ в плоскости Oxy , ориентированный «влево» от направления $\mathbf{u}$ (поворот на $+90^\circ$ в плоскости экрана север-восток), задаётся

$$\mathbf{n} = (-u_y, u_x)^\top \quad (2)$$

(функция `_normal_left_xy`).

Проекция положения i -го дрона на ось, проходящую через точку A в направлении $\mathbf{u}$, определяется скаляром

$$s_i = (p_i - A) \cdot \mathbf{u}. \quad (3)$$

В коде это `_projection_s(endpoint_a, u, p)` с `endpoint_a = pos_l`.

Поперечное (подписанное) отклонение от прямой AB удобно записать как

$$e_{\text{cross}, i} = (p_i - A) \cdot \mathbf{n}; \quad (4)$$

оно совпадает со знаковым расстоянием до прямой, проходящей через A вдоль $\mathbf{u}$, если $\|\mathbf{n}\| = 1$.

3 Движение якорей и целевые точки в симуляторе

После инициализации и обмена координатами для концевых бортов вычисляются *целевые* точки A^* , B^* в той же общей плоскости (`_anchor_targets_from_initial_segment`). Пусть L — параметр длины сегмента `segment_length` (в метрах); $L_{\text{half}} = L/2$. Если в момент расчёта якоря находятся в скорректированных точках с координатами (x_A, y_A) и (x_B, y_B) , то

$$A^* = (x_A, y_A - L_{\text{half}}, z_\star)^\top, \quad (5)$$

$$B^* = (x_B + L_{\text{half}}, y_B, z_\star)^\top, \quad (6)$$

где $z_\star$ — заданная высота удержания (в коде соответствует отрицательной высоте NED после взлёта).

Каждый якорный борт переводится к своей цели с помощью ПИД-закона по ошибке положения в горизонтальной плоскости: сначала ошибка $(e_x, e_y)^\top = (x^* - x, y^* - y)^\top$ в NED преобразуется в «вперёд-вправо» по текущему рысканию (`_ned_to_body_forward_right`), затем по осям корпуса вычисляются выходы двух регуляторов (`endpoint_loop`, `_move_towards_com`). После достижения окрестности цели (порог `ENDPOINT_REACHED_THRESH_M`) команды сбрасываются в нейтраль. Поэтому корректно говорить, что концы цепи *подводятся* к A^* и B^* и далее задают текущую геометрию отрезка AB для внутренних агентов; активное ПИД-удержание цели после срабатывания порога в этом цикле уже не выполняется.

4 Закон управления внутренних дронов

Внутренний цикл (`internal_chain_anatoliy_loop`) с частотой `CONTROL_HZ` на каждом шаге заново вычисляет $\mathbf{u}$, $\mathbf{n}$, A по текущим положениям концевых бортов, собирает множество «видимых» соседей в круге радиуса R_{vis} (`r_vis`) и формирует горизонтальный вектор комбинированного ускорения/направления $\mathbf{c}$ в NED, который затем поворачивается в корпус, демпфируется по измеренной скорости и преобразуется в `RC_OVERRIDE`. Если положения якорей ещё не получены или в пределах видимости оказалось меньше двух соседей, борт получает нейтральные команды и ждёт следующего шага обновления.

Ниже $k_{\text{along}} = \text{SPACING_ALONG_K}$, $k_{\text{cross}} = \text{SPACING_CROSS_K}$, $w_A = \text{ANATOLIY_COMB_WEIGHT}$, $w_{\text{cross}} = \text{W_CROSS_M}$.

4.1 Равномерное распределение вдоль AB

Все борты с известными позициями упорядочивают по возрастанию s_i (при равенстве s — по номеру). Для внутреннего дрона i , у которого есть и левый, и правый сосед в этом порядке, вводится продольная ошибка равномерности

$$e_{\text{along}, i} = \frac{s_{i^-} + s_{i^+}}{2} - s_i, \quad (7)$$

где i^- , i^+ — соседние в отсортированном списке идентификаторы, $s_{i^\pm}$ — их проекции. Если соседа с одной из сторон нет, в реализации $e_{\text{along}, i} = 0$ (цепочка не дополняется искусственными границами по s для крайних внутренних в особом виде — крайние якоря задают геометрию через A, B). Величина $e_{\text{along}, i}$ ограничивается по модулю константой `SPACING_E_ALONG_CLAMP_M`.

Вклад в горизонтальную команду, отвечающий за выравнивание шага вдоль цепи:

$$\mathbf{g}_{\text{geom}} = k_{\text{along}} e_{\text{along}, i} \mathbf{u} - k_{\text{cross}} e_{\text{cross}, i} \mathbf{n}. \quad (8)$$

Первое слагаемое тянет проекцию s_i к середине между соседями по s , что в целевом режиме соответствует выравниванию интервалов между соседними дронами вдоль AB .

4.2 Подавление отклонения от прямой AB

Второе слагаемое в $\mathbf{g}_{\text{geom}}$ использует $e_{\text{cross}, i}$: оно направлено вдоль $-\mathbf{n}$ с коэффициентом k_{cross} , то есть уменьшает поперечное смещение относительно прямой через A вдоль $\mathbf{u}$. Величина $e_{\text{cross}, i}$ также ограничивается по модулю (`SPACING_E_CROSS_CLAMP_M`).

4.3 Вспомогательная локальная поправка по видимым соседям

Для каждого другого дрона j в пределах видимости вводятся относительные координаты в базисе $(\mathbf{u}, \mathbf{n})$:

$$\Delta p_{ij} = p_j - p_i, \quad a_{ij} = \Delta p_{ij} \cdot \mathbf{u}, \quad c_{ij} = \Delta p_{ij} \cdot \mathbf{n}.$$

По множеству $\{(a_{ij}, c_{ij})\}$ выбираются четыре расстояния до ближайшего соседа в каждом из четырёх «квадрантов» знаков (a, c) , аналогично разложению вдоль/поперёк

в сценарии коридора: d_a^+ , d_c^+ , d_a^- , d_c^- (в коде `_nearest_along_cross`). Для каждой величины хранится флаг наличия соседа в соответствующем направлении ($f_a^\pm$, $f_c^\pm$).

Функция подстановки «расстояние или зазор» (как g в `corridor_sweeping`) задаётся

$$g(d, w, f) = \begin{cases} d, & f = \text{истина (сосед найден)}, \\ w, & f = \text{ложь}. \end{cases} \quad (9)$$

Для продольной оси в реализации при отсутствии соседа подставляется $w = 0$; для поперечной — $w = w_{\text{cross}}$.

Пусть $\xi(\cdot) = \tanh(\cdot)$. Скоростные члены в законе коридора в симуляторе положены в нуль ($v_{\text{along}} = v_{\text{cross}} = 0$), чтобы избежать дрейфа сигнала от шумной телеметрии скорости. Тогда вспомогательные скаляры принимают вид

$$\sigma_a = -\xi(g(d_a^+, 0, f_a^+)) + \xi(g(d_a^-, 0, f_a^-)), \quad (10)$$

$$\sigma_c = -\xi(g(d_c^+, w_{\text{cross}}, f_c^+)) + \xi(g(d_c^-, w_{\text{cross}}, f_c^-)). \quad (11)$$

Вектор вспомогательной поправки в плоскости:

$$\mathbf{a} = \sigma_a \mathbf{u} + \sigma_c \mathbf{n}. \quad (12)$$

Итоговый горизонтальный вектор направления до поворота в корпус и демпфирования:

$$\mathbf{c} = \mathbf{g}_{\text{geom}} + w_A \mathbf{a}. \quad (13)$$

Далее $\mathbf{c}$ переводится в оси «вперёд–вправо» по рысканию (`_ned_to_body_forward_right`), из него вычитается демпфирование, пропорциональное проекции скорости в тех же осях (`INTERNAL_BODY_V_DAMP`), ошибки по осям масштабируются в приращения PWM (`INTERNAL_RC_KP`, ограничения и `_slew_int`), и отправляется `send_rc_override`.

Если и геометрическая ошибка мала (мёртвые зоны `SPACING_E_DEADBAND_M`), и $|\sigma_a|$, $|\sigma_c|$ ниже `SIGMA_DEADBAND`, борт получает нейтральные стики. Утверждения о строгой сходимости к равновесию здесь не формулируются: корректно говорить о *целевом* выравнивании вдоль AB и о роли $\mathbf{a}$ как дополнительной локальной нелинейной подстройки плотности.

5 Соответствие формул строкам в `linear_chain2.py`

- Общие координаты и сдвиг по y : `_my_position_common`, `_did_offset_y`; обмен — `coordinate_exchange_loop`.
- $\mathbf{u}$, $\mathbf{n}$, s_i , e_{cross} : `_unit_vector_xy`, `_normal_left_xy`, `_projection_s`, блок `_spacing_errors_proc`.
- e_{along} : сортировка `ids_sorted` по s , формула $0.5 * (s_{\text{lid}} + s_{\text{rid}}) - s_{\text{me}}$.
- Цели якорей A^* , B^* : `_anchor_targets_from_initial_segment`; ПИД концов — `endpoint_loop`, `_move_towards_common_with_pid`.
- $d_a^\pm$, $d_c^\pm$, флаги: `_nearest_along_cross`; g , ξ : `_g`, `_xi`; σ_a , σ_c : `_sigmas_segment_frame`.
- $\mathbf{g}_{\text{geom}}$, $\mathbf{a}$, $\mathbf{c}$: переменные `gx`, `gy`, `ax`, `ay`, `comb_x`, `comb_y` с весами `SPACING_*`, `ANATOLIY_COMB_WEIGHT`.
- Корпус и PWM: `_yaw_rad`, `_ned_to_body_forward_right`, `_velocity_xy_ned`, `_pwm_axis`, `_slew_int`, `send_rc_override`.

Простыми словами (для устного сопровождения)

Два крайних дрона — это «гвозди»: они уезжают на заранее заданные точки и задают, где должна проходить линия. Все остальные смотрят на эту линию как на дорожку: каждый старается встать ровно посередине между соседями вдоль дорожки и не уезжать вбок от неё. Дополнительно каждый учитывает, кто стоит рядом в поле зрения, через мягкую нелинейную функцию — это мелкая подправка, чтобы не толкаться и не слипаться, а не отдельная «логика коридора». Команда в итоге переводится в привычные для пилота наклоны коптера; скорость слегка гасится, чтобы не раскачивать звено.